QDay Prize Submission Brief

End-to-End Compilable Quantum Elliptic Curve Discrete Logarithm

Diego Polimeni
diego.polimeni.work@gmail.com

Approach

We implement **Shor's algorithm for the Elliptic Curve Discrete Logarithm Problem (ECDLP)** using **Qrisp**, a high-level quantum programming language built on top of Qiskit. The repository submission is a *dynamic* implementation: given a generator point G and public key $P = kG$ on an elliptic curve $y^2 = x^3 + 7 \pmod{p}$, the same code path can be specialized, compiled, and profiled for each concrete curve instance.

The core algorithm prepares two n-qubit QPE registers in superposition, performs $2n$ controlled EC point additions (multiplying by both G and P), applies inverse QFT, and extracts k from the measurement outcomes via continued-fraction expansion. This follows the standard Shor ECDLP construction described in our paper [1].

Techniques and Design Choices

Static circuit compilation. As detailed in [1], we realize all EC arithmetic—point addition, point doubling, Kaliski modular inversion, Montgomery modular multiplication—as compilable quantum circuits using Qrisp's `QuantumModulus` type and in-place modular arithmetic primitives. Every operation is decomposed into elementary gates (H, CNOT, T, $T^\dagger$) at compile time.

Dynamic compilation via JAX (`jasp`). Beyond the static approach, we introduce **JAX-traceable dynamic compilation** through Qrisp's `jasp` subsystem. The quantum algorithm is traced through `jax.jit`, enabling dynamic circuit generation, loop unrolling, and optimization. This makes the implementation directly compilable for each curve instance instead of requiring a separate hand-specialized static circuit for every target size.

BigInteger support. To handle prime fields exceeding the native 64-bit JAX integer limit, we developed a `BigInteger` class—a fixed-width array of 32-bit limbs registered as a JAX pytree node. This enables compilation for arbitrarily large primes (up to 256-bit and beyond) while remaining fully JAX-traceable. All classical computation uses Python `int` values; `BigInteger` values appear only in `QuantumModulus` register construction and internal operator dispatch. In practice, this means the same dynamic implementation can be compiled and resource-profiled from the QDay prize curves through the 256-bit setting.

Scalable resource estimation. Every controlled EC addition in the Shor loop has *identical gate structure* regardless of the classical point—only the modulus bit-width matters. We exploit this by counting gates for a *single* controlled addition via Qrisp's `@count_ops` decorator and computing:

$$\text{Total gates} = 2n \times (\text{single EC addition}) + \text{overhead}$$

where overhead covers Hadamard preparation, initial encoding, inverse QFT, and measurement. This makes the dynamic, BigInteger-enabled implementation easy to profile, and keeps 256-bit resource estimation feasible on standard hardware without tracing 512 full EC additions.

To make the repository structure explicit, we separate the two resource-estimation use cases into two notebooks: `resource_estimation_all_curves.ipynb` compiles all QDay prize curves (4–21 bit), while `EC_discrete_log_compilation.ipynb` focuses on larger generated curves and the scaling path up to 256-bit.

Requirements and Execution

- **Software**: Python 3.11+, Qrisp (development branch with PR #495 for `QuantumModulus/BigInteger` fixes and compatibility support, and PR #505 for `count_ops` profiling optimization, both pending merge), JAX, NumPy, SymPy, Matplotlib.
- **Hardware**: Standard laptop/desktop. Resource estimation for 256-bit takes minutes; full simulation is limited to small curves ($\leq$5-bit) due to exponential state-space scaling.
- **Execution**: Install via `pip install -r requirements.txt`. Run `resource_estimation_all_curves.ipynb` for the full QDay prize curve set, and run `EC_discrete_log_compilation.ipynb` for compilation and resource estimation on larger curves up to 256-bit.
- **Validation**: Unit tests (`pytest tests/ -v`) verify correctness of all quantum primitives against classical reference implementations using Qrisp's boolean simulator.

Potential Drawbacks

- **No fault-tolerant hardware execution**: Current quantum hardware cannot execute circuits of this depth. The submission demonstrates correctness and compilation feasibility, not hardware execution.
- **No T-gate optimization**: The current compilation does not apply T-gate reduction techniques (e.g., T-factory scheduling, magic state distillation budgeting). Gate counts represent unoptimized decompositions.
- **Qrisp dependency on development branches**: The implementation requires two PRs not yet merged into Qrisp's main branch.
- **Static point encoding**: The current approach encodes classical EC points statically. Windowed or semi-classical QFT variants could reduce qubit count but are not yet implemented.

Acknowledgments

I thank René Zander and Raphael Seidel for helpful discussions and for reviewing this work.

References

[1] D. Polimeni, R. Seidel. *End-to-end compilable implementation of quantum elliptic curve logarithm in Qrisp.* arXiv:2501.10228 (2025).